Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	TE	Semester:	VI
Course Code:	CSL601	Course Name:	Data Analytics and Visualization Lab

Name of Student:	**Shravani S Raut**
Roll No.:	**40**
Experiment No.:	**04**
Title of the Experiment:	**Time Series Analysis in Python**
Date of Performance:	**17/02/2026**
Date of Submission:	**24/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Meet Expect Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms. Odilia Gonsalves

Signature:

Date:

Experiment No. 4

Aim: - Implement Time Series Analysis for Weather Data in Python Programming.

Objective: -

To understand the components of a time series.

To identify stationarity in data.

To apply AR, MA, ARMA models.

To forecast future values based on historical observations.

Description: -

Time Series Analysis is a statistical technique used to analyze data points collected or recorded at successive time intervals (daily, monthly, yearly, etc.). The primary goal is to identify patterns and forecast future values.

A time series is different from general data because order and time dependency matter.

Example:

Daily rainfall measurements

Monthly temperature records

Annual production data

Components of Time Series

1. **Trend (T)**
   Long-term increase or decrease in data.

2. **Seasonality (S)**
   Repeating pattern at regular intervals (monthly, yearly).

3. **Cyclic Component (C)**
   Fluctuations occurring over long, irregular periods.

4. **Irregular/Random Component (I)**
   Unpredictable variations.

Time Series Models

1.Autoregressive Model (AR)

Uses past values to predict current value.

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \epsilon_t$$

2. Moving Average Model (MA)

Uses past error terms.

$$Y_t = \mu + \theta_1 \epsilon_{t-1} + \epsilon_t$$

3. ARMA model

It combines both AR and MA models.

Procedure

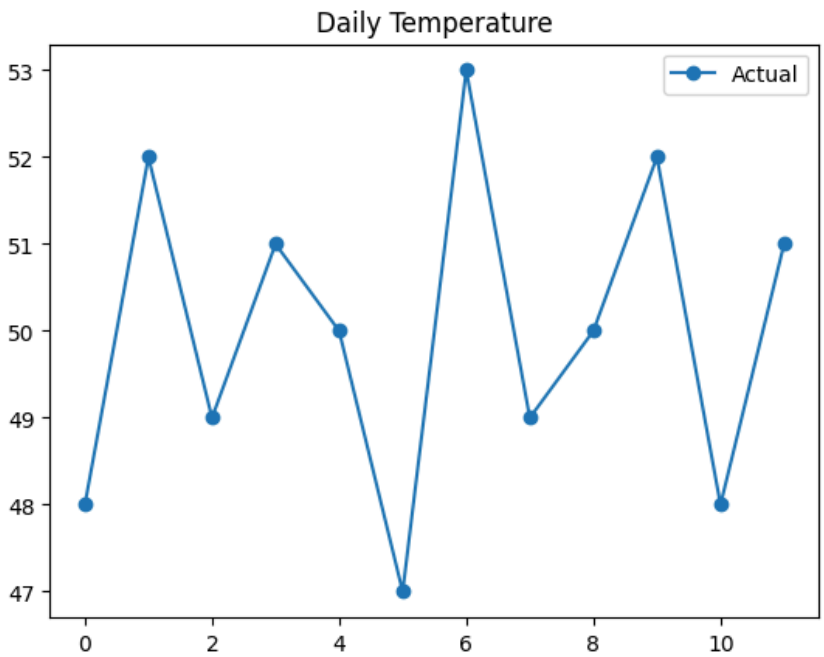
1. Collect time-based data.
2. Plot the time series.
3. Check stationarity.
4. Identify model using ACF and PACF.
5. Fit AR/MA/ARMA model.
6. Forecast future values.
7. Evaluate model accuracy.

Program: -

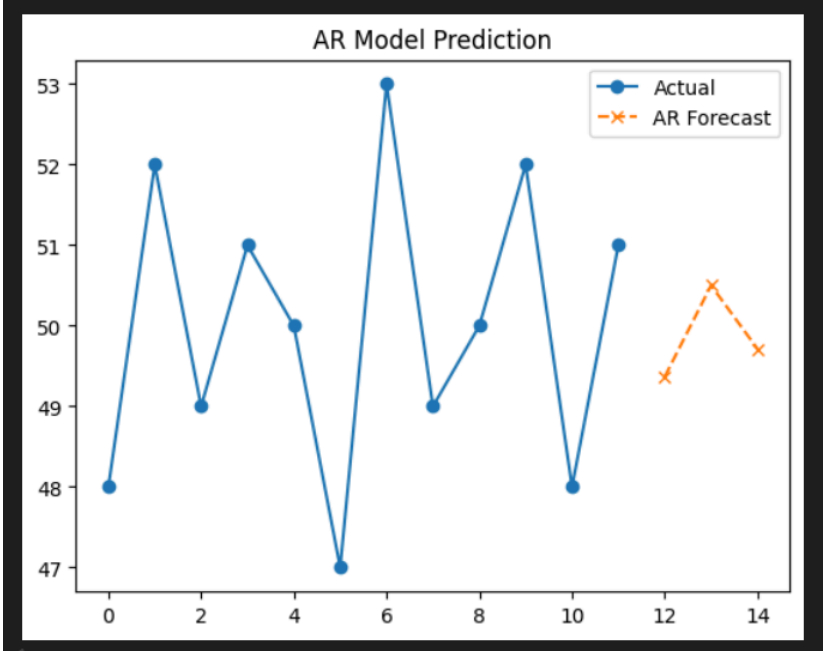
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error, mean_squared_error

# 1. Collect data
data = [48, 52, 49, 51, 50, 47, 53, 49, 50, 52, 48, 51]
df = pd.Series(data)

# 2. Plot original data
df.plot(title="Daily Temperature", marker='o', label='Actual')
plt.legend()
plt.show()

# Define models to run: Name and (p, d, q) order
# AR: (1,0,0) | MA: (0,0,1) | ARMA: (1,0,1)
models = {
    "AR": (1, 0, 0),
    "MA": (0, 0, 1),
    "ARMA": (1, 0, 1)
}

print("--- Forecast Results ---")

for name, order in models.items():
    # 5. Fit model
    model_fit = ARIMA(df, order=order).fit()
```

```
    # 6. Forecast next 3 days
    forecast = model_fit.forecast(steps=3)
    print(f"{name} Forecast: {forecast.values}")

    #7 . Plot graphs
    plt.figure()
    plt.plot(df, label='Actual', marker='o')
    plt.plot(range(len(df), len(df)+3), forecast, label=f'{name} Forecast', marker='x', ls='--')
    plt.title(f"{name} Model Prediction")
    plt.legend()
    plt.show()


results = {}

print("\n--- Model Accuracy Comparison ---\n")

for name, order in models.items():

    # 4. Fit model on training data
    model = ARIMA(train, order=order)
    model_fit = model.fit()

    # 5. Forecast for test period
    forecast = model_fit.forecast(steps=len(test))

    # 6. Calculate accuracy metrics
    mae = mean_absolute_error(test, forecast)
    mse = mean_squared_error(test, forecast)
    rmse = np.sqrt(mse)

    results[name] = rmse

    print(f"{name} Model:")
    print(f"MAE  = {mae:.4f}")
    print(f"MSE  = {mse:.4f}")
    print(f"RMSE = {rmse:.4f}")
    print("----------------------------")

# 7. Find best model (lowest RMSE)
best_model = min(results, key=results.get)
print(f"\nMost Accurate Model: {best_model} (Lowest RMSE)")
```

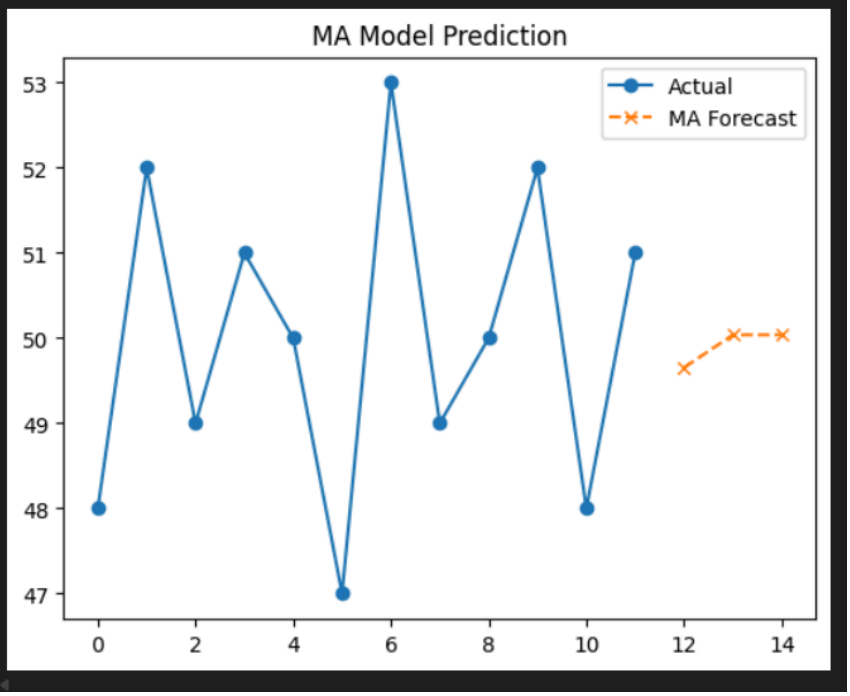
Output:



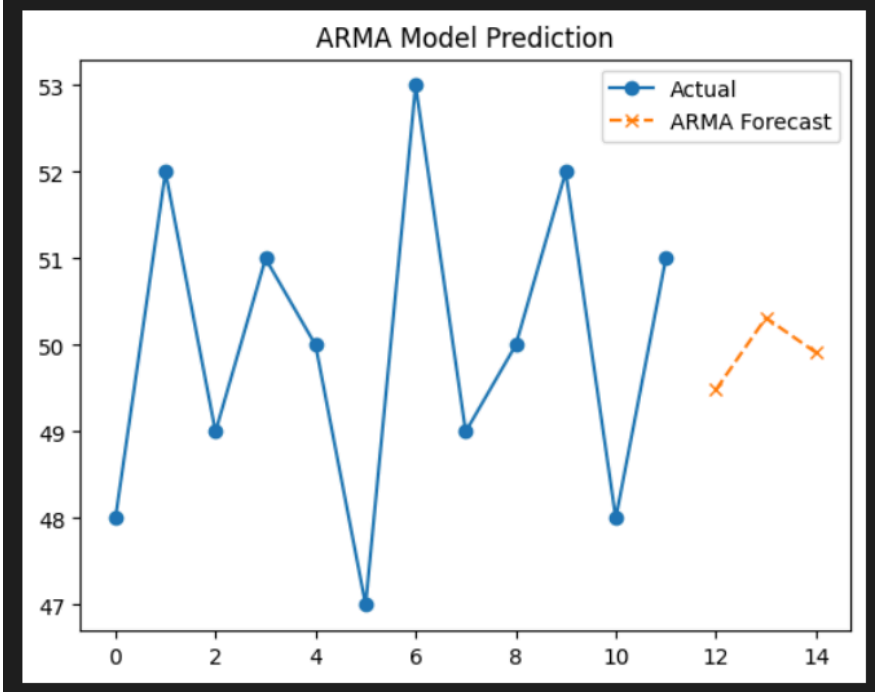

```
--- Forecast Results ---
AR Forecast: [49.36751446 50.50143936 49.71381482]
```



```
MA Forecast: [49.65382597 50.03845863 50.03845863]
```




```
ARMA Forecast: [49.48390142 50.31532564 49.91450481]
```




```
AR RMSE: 1.7484
MA RMSE: 1.7492
ARMA RMSE: 1.7163

Most Accurate Model: ARMA (Lowest RMSE)
```

Conclusion:

By comparing AR, MA, and ARMA models, this experiment demonstrates how historical dependencies (p) and residual errors (q) are quantified to identify the most stable pattern in weather fluctuations. The selection of the model with the lowest RMSE ensures that the final 3-day temperature forecast is mathematically optimized to minimize prediction error based on past observations.